

The Automated LLM Speedrunning Benchmark: Can an Agent Reproduce Known nanoGPT Speedups?

A deliberately deflationary benchmark. Before asking whether an AI research agent can *discover* new ML, ask whether it can *reproduce* an already-published speedup when handed the previous code and a written description of the change. Meta FAIR turns the community-driven NanoGPT Speedrun — 19 successive record-breaking edits to a GPT-2 124M training script — into 19 reproduction tasks, and finds that frontier reasoning models plus SoTA agent scaffolds recover **less than 20% of the human speedup with no hints, and plateau around 40–46% even when given a paper-like write-up of exactly what to do**. Reproduction, the necessary-but-not-sufficient floor beneath autonomous discovery, is not solved.

Background

Primary paper: The Automated LLM Speedrunning Benchmark: Reproducing NanoGPT Improvements (Bingchen Zhao, Despoina Magka, Minqi Jiang, Xian Li, Roberta Raileanu, Tatiana Shavrina, ... Jakob Foerster, Yoram Bachrach — Meta FAIR + University of Edinburgh + University of Oxford; v2 dated 2 Jul 2025). Code: github.com/facebookresearch/llm-speedrunner. Senior author Roberta Raileanu (also led MLGym) has since moved to Google DeepMind's Open-Endedness team; the paper is a direct sibling of MLGym in the FAIR "AI research agent" line.

The benchmark sits on top of a four-step lineage you already have most of:

1. **nanoGPT** (Karpathy, 2023, github.com/karpathy/nanoGPT) — the minimal, hackable GPT-2 reproduction *you trained your own 124M model with*. This is the literal starting script (`train_gpt2.py`).
2. **The NanoGPT Speedrun** (Keller Jordan et al., 2024a, github.com/KellerJordan/modded-nanogpt) — a community competition to *minimize wall-clock time* to train that GPT-2 124M, on a fixed **8×H100 node**, down to a **validation cross-entropy loss of 3.28 on FineWeb**. Since June 2024 the community drove this from **45 minutes to under 3 minutes** (as of May 2025), across **21 successive records**, each shipping a training script, a measured time, and a public write-up of the change.
3. **The Muon optimizer** (Keller Jordan et al., 2024b, kellerjordan.github.io/posts/muon) — the single most consequential speedrun innovation (record #3), later shown to help train much larger LLMs (Liu et al. 2025a, *Muon is scalable*; Shah et al. 2025, *Practical Efficiency of Muon*). Defined in Key Terms.
4. **FineWeb** (Penedo et al., 2024, OpenReview n6SCkn2QaG) — the web-text corpus whose held-out split defines the 3.28 loss target.

Where it sits in this repo's autonomous-research thread. This is a *reproduction* benchmark, and its message rhymes with the field's other negative results. See [landscape-2025-2026.md](#) (Section 5 #16). It is the ML-engineering-flavored companion to the Ideation–Execution Gap: that paper shows LLM *ideas* lose their edge once executed; this one shows LLM *agents* can't even execute someone else's already-validated idea reliably. Both point at the same crack — generation and pitch-scoring are cheap; faithful execution and verification are where the wheels come off. Cross-reference the execution-grounding argument in [execution-grounded-auto-research-guide.md](#) and the recursive-self-improvement framing in [iclr-2026-rsi-workshop.md](#).

What problem does it solve

Every "AI scientist" benchmark faces a measurement problem: *discovery* is expensive to grade (you need novel, correct, useful results, and no ground truth exists) and rare enough that a handful of cherry-picked successes prove little. The authors sidestep this with a sharp reframing:

DISCOVERY (what everyone wants to measure)	REPRODUCTION (what this measures)
"invent a faster training method" • no ground truth, no clean metric • success is rare + contestable • can't tell luck from skill	"here is record R_i and a description of the change that produced R_{i+1} ; re-implement it" • ground-truth code + time exist • single scalar metric (wall time) • cheap: records run in minutes
reproduction is a LOWER BOUND on discovery if an agent cannot re-derive a KNOWN speedup given the paper describing it, it certainly cannot find an UNKNOWN one. Necessary, not sufficient.	

Why the NanoGPT Speedrun is an unusually good substrate for this:

- **Ground-truth code + times exist** for every record, so grading is an objective, deterministic wall-clock comparison on fixed hardware — no LLM judge required for the headline metric.
- **Records run fast** (minutes, not the GPU-weeks of a real frontier run), so you can afford thousands of agent runs.
- **It is a genuine research arc, not a toy** — the 19 changes span the full spectrum of real LLM-training work: new optimizers, architecture surgery, attention-kernel swaps, numerical-precision tricks, and hyperparameter tuning. Each is a real innovation a human researcher actually shipped.
- **It is cumulative and sequential** — unlike MLE-bench / PaperBench (unrelated one-off tasks), the records form one compounding chain, so you can also test whether an agent can build record $N+1$ *on top of its own reproduction* of record N (the cumulative experiment, below).

Key Terms

NanoGPT Speedrun / modded-nanoGPT: the community competition (#2 in Background). A "record" is a `train_gpt.py` that reaches val-loss 3.28 faster than the prior record on $8\times H100$. Not a leaderboard of models — a leaderboard of *training scripts*, ranked by wall-clock time.

Muon (MomentUm Orthogonalized by Newton-schulz): the record-#3 optimizer. Standard momentum/SGD produces an update matrix G for each 2D weight; Muon replaces G with its nearest **semi-orthogonal matrix** (the UV^T from G 's SVD) — but instead of a costly SVD it runs a few steps of a **Newton-Schulz iteration**, a matrix-polynomial recurrence that drives all singular values toward 1 while staying in fast, bf16-friendly matmuls. Intuition vs your AdamW: AdamW scales each weight's update by a per-coordinate second-moment estimate; Muon instead *orthogonalizes the whole update matrix*, equalizing its singular values so no direction dominates. Muon is applied only to 2D hidden weights; embeddings, the LM head, and 1D gains/biases stay on AdamW.

Newton-Schulz orthogonalization: the iteration $X \leftarrow a \cdot X + b \cdot (XX^T)X + c \cdot (XX^T)^2X$, repeated ~ 5 times on the normalized update, which converges X to a matrix with (approximately) unit singular values — a cheap, differentiable, GPU-parallel stand-in for the orthogonal factor of an SVD.

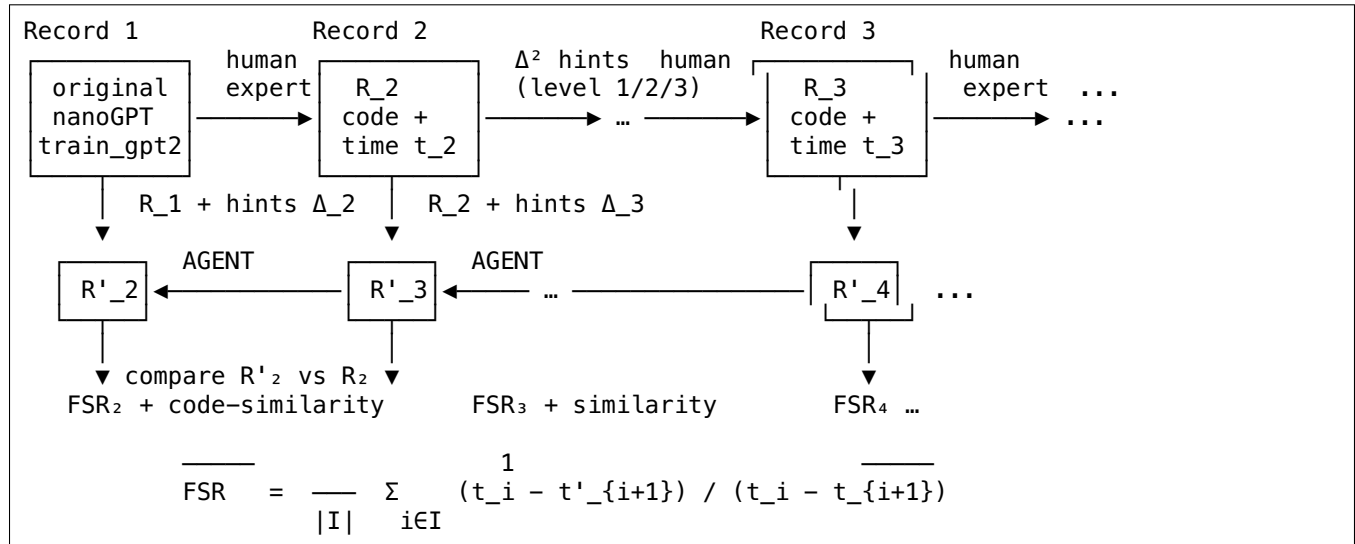
Scaffold / agent harness: the *program* that wraps an LLM into a research loop — it prompts the model to edit code, runs the code, summarizes the result, and decides what to try next. The paper's term for "(specific LLM) + (specific scaffold)" is a **research agent**. The benchmark holds the scaffold family fixed and varies both the LLM and the scaffold's search strategy.

FSR — Fraction of Speedup Recovered: the core metric (below). The share of the *human's* wall-clock improvement on a transition that the agent reproduces. 1.0 = matched the human record; 0 = no improvement over the starting script; <0 = the agent's edit made it slower (or broke it).

Reproduction task vs optimization task: with hints ($m \neq \{\emptyset\}$) the agent must reproduce the *specific* next record $\rightarrow$ graded by FSR against that record. Without hints ($m = \{\emptyset\}$) it must just produce *any* faster script from the same starting point $\rightarrow$ graded by raw wall time and FSR.

The benchmark construction

For each transition from record $R_{\{i-1\}}$ to R_i ($i = 2 \dots 21$), the paper defines a task. One transition — $i = 7$, the pure **PyTorch 2.5.0 upgrade** (record #7, 12.0 min) — is dropped because its speedup owes nothing to code changes. That leaves **19 tasks**. Each task is a tuple $(R_{\{i-1\}}, R_i, t_i, m)$: the starting script, the target script, the target's wall time, and a hint subset $m \subseteq \{\emptyset, 1, 2, 3\}$.



The FSR metric (Eq. 1–2). For a single transition where the human went from time t_i to $t_{\{i+1\}}$, and the agent's reproduced script $R'_{\{i+1\}}$ reaches the 3.28 loss in $t'_{\{i+1\}}$:

$$FSR_i = \frac{t_i - t'_{\{i+1\}}}{t_i - t_{\{i+1\}}} \quad FSR\text{-bar} = \frac{1}{|I|} \sum_{i \in I} FSR_i$$

$|I| = 19$. The denominator is the human's speedup on that transition; the numerator is the agent's. Note the metric can go **negative** — if the agent's "improvement" is slower than the starting script (a common R1 outcome under hints), or if it introduces a bug and never reaches 3.28 within the 60-minute cap.

The 19 tasks (Appendix E, Table E.1). The full ladder, which doubles as a compact tour of what actually made GPT-2 training $15\times$ faster:

Task	Orig#	Transition	Rec.time	Change	Category
–	1	–	45 min	llm.c baseline	Baseline
1	2	#1 → #2	31.4 min	Tuned learning rate & rotary embeddings	Embeddings
2	3	#2 → #3	24.9 min	Introduced the Muon optimizer	Optimizer
3	4	#3 → #4	22.3 min	Muon improvements	Optimizer
4	5	#4 → #5	15.2 min	Pad embeddings, ReLU ² , zero-init proj, QK-norm	Architecture
5	6	#5 → #6	13.1 min	Distributed the overhead of Muon	Parallelization
–	7	–	12.0 min	Upgraded PyTorch 2.5.0 (EXCLUDED)	Framework
6	8	#6 → #8	10.8 min	Untied embedding and head	Architecture
7	9	#8 → #9	8.2 min	Value/embed skip connections, momentum warmup, logit softcap	Architecture
8	10	#9 → #10	7.8 min	Bfloat16 activations	Data Type
9	11	#10 → #11	7.2 min	U-net skip connections & double lr	Architecture
10	12	#11 → #12	5.03 min	1024-ctx dense attn → 64K-ctx FlexAttention	Attention
11	13	#12 → #13	4.66 min	Attention window warmup	Attention
12	14	#13 → #14	4.41 min	Value Embeddings	Embeddings
13	15	#14 → #15	3.95 min	U-net value embeddings, code optimizations	Embeddings
14	16	#15 → #16	3.80 min	Split value embeddings, block sliding window	Embeddings
15	17	#16 → #17	3.57 min	Sparsify value embeds, better rotary, drop an attention layer	Embeddings
16	18	#17 → #18	3.4 min	Lower logit softcap from 30 to 15	Hyperparam
17	19	#18 → #19	3.142 min	FP8 head, offset logits, lr decay to 0.1	Data Type
18	20	#19 → #20	2.992 min	Merged QKV, long-short attention, batched Muon	Attention
19	21	#20 → #21	2.933 min	Reduced batch size	Hyperparam

(Beware the two index systems: the benchmark's **task index 1–19** ≠ the speedrun's **original record #**, which skips the excluded PyTorch upgrade. "Record 2" in the paper's figures usually means task index 2 = the Muon transition.)

The three hint tiers

The knob that makes this a controlled study is *how much information about the change the agent gets*. Each transition ships three hint levels, all first drafted by DeepSeek-R1 from the real git diff + official changelog, then manually verified and corrected:

Δ ¹	Level 1 – pseudocode	high-level pseudocode of the code change; algorithmic logic, no implementation detail
Δ ²	Level 2 – text	natural-language prose description of the improvements
Δ ³	Level 3 – mini-paper	a formal paper-like write-up (abstract, method, code snippets, hyperparameter table, results) – the most verbose

Hints compose: the paper evaluates six regimes — {0} (none), {1}, {2}, {3}, {1+2}, {1+2+3}. This is the study's cleanest axis: it interpolates from "figure it out yourself" to "here is a mini-paper telling you exactly what to do," so a flat FSR curve across hint levels would mean the bottleneck is *implementation*, not *information*.

A real hint, all three tiers (Appendix F, task 1, #1 → #2). The first transition bundles rotary embeddings + a trapezoidal LR schedule + per-parameter gradient normalization + attention scaling + dropping the wpe positional-embedding matrix + doubling batch size (32→64). The same change rendered at each tier:

LEVEL 1 (pseudocode)	LEVEL 2 (text)	LEVEL 3 (mini-paper)
<pre>class RotaryPositionEmbedding: precompute inv freqs cache cos/sin ... def apply_rotary_emb(q,k,...): rotate halves, q1*cos+q2*sin class Block: attn_scale = 1/sqrt(2*n_layer) batch_size 32 → 64</pre>	<pre>"1. Rotary Positional Embeddings: Replaced standard positional embeddings ... 2. LR: 0.0015 → 0.0018 (3×), trapezoidal schedule ... 3. grad = grad/(norm+1e-6) batch 262K → 524K tok"</pre>	<pre>"# Efficient Training of GPT-style Models ... ## Abstract We present ... 2× faster ... ## 2. Method 2.2 Trapezoidal LR: lr = base_step/256 (warmup) Param Orig Mod Batch 32 64 ..."</pre>

The mini-paper even fabricates a plausible results table ("d12 (124M): 3.21 → 3.09 loss") — a reminder that the *hint*, not just the agent, is LLM-authored.

Numerical walkthrough — the Muon task (#2 → #3)

Take the marquee transition (task index 2): introducing **Muon**. The agent is handed record #2's script (a 31.4-minute run using AdamW + rotary embeddings), plus a hint describing "replace the optimizer for 2D hidden weights with momentum + Newton–Schulz orthogonalization; keep AdamW for embeddings and head." Its job: emit R'_3 that hits val-loss 3.28 as fast as possible. Ground truth: the human's record #3 runs in **24.9 min**.

Endpoints for this task:
t_i = t₂ = 31.4 min (starting script the agent is given)
t_{i+1} = t₃ = 24.9 min (human target; the "24.9" row in Table E.1)
human speedup = 31.4 - 24.9 = 6.5 min ← the FSR denominator

$$FSR_2 = \frac{31.4 - t'_{i+1}}{6.5}$$

Four illustrative agent outcomes:

agent time t'	what happened	FSR ₂	
24.9 min	correct, matched the human	1.00	
28.0 min	correct Muon, but slower Newton–Schulz impl	0.52	(3.4 / 6.5)
31.4 min	no working change / reverted (buggy Muon)	0.00	
34.0 min	"improvement" that regressed throughput	-0.40	(-2.6 / 6.5)
– (no 3.28)	bug: never reaches target in 60-min cap	≤ 0	

The reproduction is *conceptually* one idea, but the failure surface is large: get the Newton–Schulz coefficients or iteration count wrong and you either don't converge (loss never hits 3.28) or you pay too much per step (slower than AdamW, FSR ≤ 0); forget to keep embeddings/head on AdamW and training destabilizes. This is why "reproduce a known result" is a real test and not bookkeeping — the description tells you *what*, and translating it into *correct, performant, numerically-stable CUDA-friendly PyTorch* is exactly where agents bleed FSR. Empirically, this task (index 2) is one of the more reproducible ones; the later multi-change records (tasks 10+) collapse toward 0.

The agent scaffolds

The harness is a generalization of **AIDE** (Jiang et al. 2025), the tree-search coding agent, with **Aider** (Gauthier 2025) doing the actual diff-based edits. Each node in the search tree is a candidate solution (a train_gpt.py + its measured metrics + an LLM summary of its run). Every step runs three stages — **implement** → **execute (on 8×H100, 60-min cap)** → **analyze** — then the tree grows by either improving the best node or debugging a buggy one.

```

def speedrun_agent(R_start, hints, M, N0, N, p_debug, D_max):
    root = Node(code=R_start, knowledge=hints) # ground-truth start + hints
    frontier = [draft_child(root) for _ in range(N0)] # N0 initial attempts

    for step in range(M): # fixed budget: M = 20 nodes
        if random() < p_debug and buggy_leaves(tree):
            parent = pick_random(buggy_leaves(tree)) # DEBUG a crashed node
            if debug_depth(parent) > D_max: continue # (bounded retries)
        else:
            parent = argmax(tree, key=lambda n: n.fsr) # IMPROVE the best node

        for _ in range(branch_factor): # N children (1 for AIDE)
            code = aider_diff_edit(parent, task_desc, hints) # IMPLEMENT
            metric = run_on_8xH100(code, timeout=60*60) # EXECUTE
            summ = llm_summarize(metric.stdout, metric.stderr) # ANALYZE
            tree.add(Node(code, metric.wall_time, summ, parent))

    return best_by_time_to_target_loss(tree) # → t', hence FSR

```

The five variants are just settings of (N0, branch, p_debug, D_max) — deliberately spanning "no iteration" to "full debug-and-branch":

Variant	Init branch	Branch	Debug prob	Max debug depth	character
Flat (BoM)	M	—	—	—	best-of-M, no iteration
Tree	1	N	0	0	branch, no debug
Forest	N0	N	0	0	wide, no debug
AIDE	N0	1	p_debug	D_max	iterate + debug, no branch
Multi-AIDE	N0	N	p_debug	D_max	branch + debug (richest)

Scale of the study. 4 models × 5 scaffolds × 6 hint regimes × 19 tasks × 3 seeds = **6,840 agent runs**, ~10 GPU-hours each (60-min cap per solution, up to 20 h per run), i.e. **6,840 × 8 H100-hours**. Baselines: N0 = 3, N = 3, p_debug = 0.5, D_max = 5, M = 20.

Results

Four LLMs — **DeepSeek-R1**, **o3-mini**, **Gemini-2.5-Pro**, **Claude-3.7-Sonnet**. Mean FSR (fraction of human speedup recovered), **Flat** / **best-of-M** scaffold, mean over 3 seeds (Table 3; the {0} column is from Fig. 4 — "all agents < 0.20 with no hints"):

Model	no hint	L1 pseudo	L2 text	L3 mini-paper	L1+L2	L1+L2+L3
o3-mini	<0.20	0.40	0.22	0.17	0.27	0.24
DeepSeek-R1	<0.20	0.13	0.10	0.13	0.25	0.30
Gemini-2.5-Pro	<0.20	0.18	0.18	0.18	0.18	0.19
Claude-3.7-Son.	<0.20	0.14	0.10	0.06	0.14	0.21

Headline reads:

- Hints are necessary.** Every model recovers < 20% of the human speedup with no hints — the single most robust finding.
- But they saturate low.** The best single number in the whole paper is **o3-mini at ~0.46** (all hints, multi-AIDE scaffold) — i.e. even handed a mini-paper spelling out the exact change, the best agent leaves **more than half** the human speedup on the table.
- Pseudocode (L1) is the most useful single hint** for o3-mini; verbose prose/mini-papers alone are *worse* than terse pseudocode.
- Models split on hint-combining.** Piling text + mini-paper onto pseudocode **degrades** o3-mini (0.40 → 0.24 flat: long context hurts it) but **helps** R1 (0.13 → 0.30 flat, and up to 0.41 combined): R1's reasoning benefits from longer prompts, o3-mini's doesn't.

5. **o3-mini is the clear leader; R1 is second with combined hints.** Best per model (any scaffold): o3-mini 0.46, R1 0.41, Gemini 0.26, Claude 0.34.

The IQM twist (Fig. 5). Averaged FSR flatters unreliable models. Under the **interquartile mean** (robust to outlier good runs), **Gemini-2.5-Pro and Claude-3.7-Sonnet collapse to ≈ 0 FSR — behind even open-weights R1.** The reason (Fig. 8): Claude generates a *large fraction of buggy nodes* (crashes), which the mean hides but the IQM exposes. o3-mini stays on top under both metrics.

Scaffold sophistication barely helps (Fig. 4, §4.2). Flat **best-of-M matches or beats** the iterated search scaffolds across individual hint levels; tree/forest (no debug) $\approx$ AIDE (with debug). **Explicit debug loops add little** — the bottleneck is the model's first-shot implementation, not its self-repair.

Later records are harder (Fig. 6). FSR and code-similarity both decay along the ladder — the early single-idea records (Muon, bf16) are far more reproducible than the late multi-change records (task 10+: merged-QKV + long-short attention + batched Muon).

Injecting extra knowledge can hurt (Table 4). Task 10 (#11 $\rightarrow$ #12) introduces **FlexAttention** (Dong et al. 2024), a PyTorch module released *after* the models' knowledge cutoff. Pasting the FlexAttention docs into context **lowered** FSR (R1 0.09 $\rightarrow$ 0.07; o3-mini 0.10 $\rightarrow$ 0.06) — recent models still fail to exploit genuinely-new external knowledge on complex tasks.

Cumulative reproduction falls off a cliff (Fig. 9). The hardest variant: let the agent build record N+1 *on top of its own reproduced* record N. Starting from ground-truth R_1, o3-mini (multi-AIDE, all hints) recovers $\sim 60\%$ for R'_2, then $\sim 20\%$ for R'_3 (vs 60% when restarted from the *ground-truth* R_2), and **by R'_4, 0% — no speedup at all.** Errors compound; the agent cannot stand on its own shoulders.

Failure modes — why agents fail even with the answer

1. Description $\rightarrow$ performant code is the wall. Flat FSR-vs-verbosity shows the bottleneck is not knowing **WHAT** to do (the mini-paper says so) but writing correct, fast, numerically-stable PyTorch that realizes it.
2. Buggy first drafts dominate. Most initially-proposed solutions crash; Claude-3.7 especially $\rightarrow$ its mean FSR looks fine but IQM ≈ 0 (Fig. 8).
3. Self-repair barely helps. Debug loops (AIDE) $\approx$ no-debug (tree/forest); best-of-M is competitive $\rightarrow$ iteration doesn't rescue a bad first shot.
4. Brittle context use. o3-mini degrades as hints get longer; R1 improves. No model robustly turns "more information" into "more FSR."
5. Can't absorb new knowledge. FlexAttention docs in-context **HURT** reproduction of the record that needs them (Table 4).
6. No compounding. Cumulative reproduction decays to 0 by the 3rd self-built record (Fig. 9) — the opposite of the recursive self-improvement dream.
7. Not (just) a memorization story. Records predate the cutoffs, yet neither R1 nor o3-mini reproduces them accurately — so the failure is generalization of engineering skill, not a missing lookup. (Disentangling the two is future work; §5 "Memorization or generalization?")

Versus alternative agent/reproduction benchmarks

The paper's own differentiation (Table 1), extended with headline numbers from landscape-2025-2026.md:

Benchmark	Reprod?	Sequential?	LLM-research?	Scaffold?	Headline result
MLE-bench (OpenAI)	No	No	No	No	75 Kaggle comps; o1-preview +AIDE $\geq$ bronze in 16.9%
PaperBench (OpenAI)	Yes	No	Partly	Yes	replicate 20 ICML'24 papers; best agent ~21%
CORE-bench (Princeton)	Yes	No	No	Yes	reproduce 90 papers; ~21% on hardest level
RE-bench (METR)	No	No	Yes	Yes	7 ML-R&D envs vs 61 humans; agents win @ 2 h, humans @ 8 h+
MLAgentBench	No	No	Partly	Yes	13 ML experimentation tasks
MLGym-bench (FAIR)	No	No	Partly	Yes	13 open-ended tasks; tunes well, invents no new algorithms
— Automated LLM Speedrunning (this)	Yes	Yes	Yes	Yes	19 nanoGPT records; <20% FSR no hint, ~40–46% with full hints

It is the only one that is **reproduction + sequential + LLM-research-specific + scaffold-shipped** at once. The sequential axis is unique: because the records form a compounding chain, it is the only benchmark that can measure whether an agent builds N+1 on its own N (and the answer, Fig. 9, is no).

Practical takeaways

- **Treat "reproduce known result" as the floor, and it's not solved.** The strongest framing in the paper: reproduction is *necessary but not sufficient* for autonomous research. Best agent $\approx$ 46% of a single known speedup with the answer in hand $\rightarrow$ discovery claims that skip this floor deserve heavy skepticism.
- **Hints roughly double FSR then plateau < 50%** — the gap is *implementation fidelity*, not ideation or information. If you're building a research agent, invest in "make the code correct and fast," not "generate more ideas."
- **Scaffold complexity is oversold here.** Tree search, debug loops, and branching bought little over best-of-M; model choice and first-shot code quality dominated. A cheap, strong baseline to beat before you add machinery.
- **Always read IQM next to mean FSR.** Averaged scores hid that Gemini/Claude were near-zero-reliable. For any agent metric with heavy-tailed per-task outcomes, report a robust aggregate.
- **A cheap, non-saturated RSI tracker.** Records run in minutes and the ceiling is far off, so this is a practical dial to watch as models improve — the intended use (§6, and the RSI-workshop framing).

Connect it to your own work

- **This benchmark trains *your* model.** The starting script is Karpathy's nanoGPT `train_gpt2.py` — the same GPT-2 124M you pretrained. Record #1's baseline is the 45-minute run; the whole ladder is "how would an expert make *your* training loop 15 $\times$ faster," decomposed into 19 auditable steps. Reading Table E.1 is a compressed masterclass in modern efficient-pretraining tricks applied to a model you already understand end-to-end.
- **Muon vs your AdamW.** You trained with AdamW (per-coordinate second-moment scaling). Muon (task 2) is the natural next concept: orthogonalize the *whole* 2D update via Newton–Schulz instead of rescaling coordinates — same role in the loop, different geometry, and now proven to scale past 124M. It's the single highest-leverage record to actually implement yourself.
- **FSR is a normalized reward; reproduction is the SFT-analog of discovery.** You know the SFT-vs-GRPO distinction: imitate a known target vs. discover behavior via reward. This benchmark tests the *engineering* analog — imitate a known, validated code change (the floor) — and shows models can't yet reliably do even that, let alone the GRPO-style open-ended search that autonomous discovery would require.

Key Papers

1. **The Automated LLM Speedrunning Benchmark** — Zhao, Magka, Jiang, ... Raileanu, Foerster, Bachrach (Meta FAIR + Edinburgh + Oxford), Jun 2025. arXiv 2506.22419 · code
2. **modded-nanoGPT / NanoGPT Speedrun** — Keller Jordan et al., 2024a. github.com/KellerJordan/modded-nanogpt (world-record history)
3. **Muon: An optimizer for hidden layers in neural networks** — Keller Jordan et al., 2024b. kellerjordan.github.io/posts/muon
4. **nanoGPT** — Andrej Karpathy, 2023. github.com/karpathy/nanoGPT
5. **Muon is scalable for LLM training** — Liu et al., 2025. arXiv 2502.16982 · **Practical Efficiency of Muon** — Shah et al., 2025. arXiv 2505.02222
6. **FineWeb** — Penedo et al., NeurIPS 2024. OpenReview n6SCkn2QaG
7. **AIDE: AI-Driven Exploration in the Space of Code** — Jiang et al., 2025. arXiv 2502.13138 · **Aider** — Gauthier, 2025. github.com/Aider-AI/aider
8. **FlexAttention** — Dong et al., 2024. arXiv 2412.05496
9. **MLGym** — Nathani et al. (FAIR), 2025. arXiv 2502.14499 (sibling benchmark)
10. Peer benchmarks: **MLE-bench** 2410.07095 · **RE-bench** 2411.15114 · **PaperBench** 2504.01848 · **CORE-bench** 2409.11363